

A LESSON IN RESEARCH METHODOLOGY

Algorithmic and Software Engineering Approaches

Two Research Traditions in Computing Science

How research is actually organised when the object of study is an algorithm, and how it changes when the object is a software system built and maintained by people.

Research Methodology in CS · Unit II · Lecture 4



The Object of Study Decides the Method

Unit II has covered how a claim is argued, proved, and inferred. This session is about how the work is organised — and computing science organises it in two quite different ways.

- 1 In the algorithmic tradition the object of study is a problem and the procedures that solve it: a mathematical object that also executes. In the software-engineering tradition it is a socio-technical practice — code, tools, process, and the people using them.
- 2 Nearly everything downstream follows from that difference: which of Lecture 2's proof modes carries the argument, which of Lecture 3's inference directions dominates, and what a reviewer will attack first.
- 3 This is not “theory versus practice”. Both traditions build artifacts, both measure, and both prove things. What differs is where the uncertainty sits — and therefore what the paper must spend its effort defending.
- 4 Most CS theses draw on both. The trouble starts when one tradition's standards are used to judge the other's work: demanding a theorem from a field study, or a user study from a lower bound.

WHERE THE UNCERTAINTY SITS

In algorithmic work the risky premise is the model. In software-engineering work it is the population.

One asks whether the machine model prices real hardware; the other asks whether these projects and these developers stand for the ones you claim about. Same slot in the argument, different contents.

Find that slot in your own work before a reviewer does.

The Two Traditions, Side by Side

Read down either column: each row is the same methodological question, answered differently.

TRADITION 1

Algorithmic

OBJECT OF STUDY

A problem, and the procedures that solve it.

CENTRAL QUESTION

How well can this problem be solved, and can it be proved?

PROOF MODE (LECTURE 2)

Mathematical, with empirical support from an implementation.

INFERENCE (LECTURE 3)

Deduction, inside a stated machine model.

WHAT A RESULT LOOKS LIKE

A bound, a guarantee, a hardness result, or a measured speed-up.

TYPICAL VENUES

SODA, ICALP, ESA, SEA, ALENEX.

TRADITION 2

Software engineering

OBJECT OF STUDY

A socio-technical system: code, tools, process, and people.

CENTRAL QUESTION

What happens when people build software, and what helps?

PROOF MODE (LECTURE 2)

Empirical, often with a demonstration artifact alongside.

INFERENCE (LECTURE 3)

Induction, from sampled projects, repositories, developers.

WHAT A RESULT LOOKS LIKE

A finding on a defined population, or an artifact plus evaluation.

TYPICAL VENUES

ICSE, FSE, ASE, MSR, EMSE.

What Counts as a Finished Contribution

Three families of CS work, and the standard each is actually held to.

FAMILY OF WORK	IT IS COMPLETE WHEN...	THE REVIEWER'S FIRST QUESTION	THE CHARACTERISTIC WEAK PAPER
Algorithmic	There is a stated model, a proved correctness argument, a derived bound or a proved impossibility — and, increasingly, an implementation measured on accepted instances.	Is the bound tight, and is the model the right one?	A new procedure with no analysis and no baseline, evaluated on instances the authors generated themselves.
Software engineering	A question is answered on a defined population, with threats to validity stated, and with the protocol or data available for someone else to re-run.	What population is this about, and would it replicate?	A tool paper whose entire evaluation is the authors demonstrating their own tool on examples they chose.
Spanning both	An artifact exists and is available, and its claimed property is both argued and measured against the strongest available alternative.	Can I run it, and was the baseline a fair one?	A prototype, a demo, a related-work list — and no comparison against anything.

Note what all three share: a stated premise, an honest comparison, and something a stranger could check. The rest is tradition-specific detail.

PART 1 → PART 2

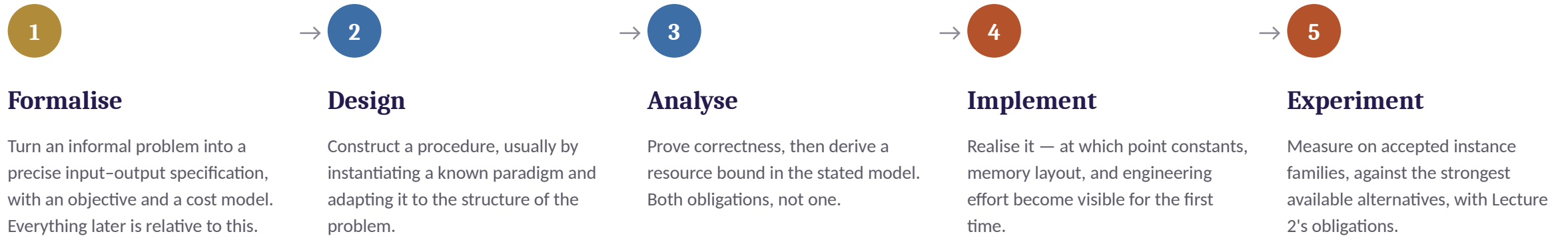
From two traditions to the first one's working method

If the object of study is a problem, what does a research programme around it actually consist of — from the first formalisation to the last measurement?

Next: the algorithmic approach — pipeline, paradigms, analysis, and the engineering loop.

The Algorithmic Research Pipeline

Five stages. The first is where most of the intellectual work lives, and the one most often skipped in student projects.



STAGES 4 AND 5 USED TO BE OPTIONAL

For decades an algorithms paper could end at stage 3, and many still legitimately do — a lower bound needs no implementation. But the experimental algorithmics community, the ALENEX and SEA venues, the DIMACS Implementation Challenges, and conference artifact evaluation have together made the full pipeline the default expectation for any paper claiming practical benefit.

The Design-Paradigm Toolkit

Eight recurring structures. As with Lecture 3's proof techniques, the skill is not executing them — it is recognising which the problem's structure admits.

Divide and conquer

Split, recurse, combine.

Analysed by recurrence. Mergesort, FFT, Strassen.

Greedy

Take the locally best choice and never revise it.

Correctness by exchange argument or matroid structure. Kruskal, Huffman.

Dynamic programming

Overlapping subproblems with optimal substructure.

Correctness by induction over subproblem order. Bellman–Ford, alignment.

Randomisation

Use random choices to buy simplicity or expected speed.

Analysed in expectation or with high probability. Quicksort, Karger, hashing.

Approximation

Give up optimality, keep a provable ratio.

For NP-hard problems. Vertex cover 2-approximation; PTAS families.

Branch and bound

Exhaustive search with sound pruning.

No worst-case gain, large practical one. ILP, SAT and CP solvers.

Local search

Iteratively improve; no guarantee at all.

Judged purely empirically. Simulated annealing, LKH for TSP.

Parameterised

Confine the exponential to a parameter k .

$f(k) \cdot \text{poly}(n)$, plus kernelisation. Treewidth, vertex-cover size.

Also standard: streaming, online, sublinear and external-memory algorithms — paradigms defined by a resource constraint rather than a control structure, each with its own analysis style.

Which Analysis Are You Reporting?

Five distinct claims, routinely confused with one another in write-ups. Each answers a different question about cost.

ANALYSIS	WHAT IT REPORTS	APPROPRIATE WHEN	WATCH OUT FOR
Worst case	A bound that holds for every input of size n , with no exceptions.	A guarantee is required, or inputs may be adversarial.	It can be dominated by inputs nobody will ever supply.
Average case	Expected cost under an assumed distribution over inputs.	That distribution is genuinely known or defensible.	The distribution is a premise — usually the weakest one present.
Amortised	Average cost per operation across a worst-case sequence.	Individual operations vary but sequences are what run.	Still a worst-case claim. Do not report it as “typical”.
Smoothed	Worst case over slightly perturbed inputs.	Explaining why an exponential method is fast in practice.	The perturbation model is itself an unverified assumption.
Competitive	Ratio to an offline optimum that knows the whole future.	Online problems where decisions cannot be revised.	The yardstick may be unreachably strong, making ratios look bad.

State which one you are claiming, in the abstract. A reader who assumes “worst case” and finds “average case under a uniform distribution” has been misled, whether or not you intended it.

When Exact and Efficient Is Out of Reach

Lecture 3's reductions tell you a problem is hard. They do not end the research — they define which four bargains remain open.

BARGAIN 1

Approximation

Give up optimality; keep a proved ratio to the optimum.

WHAT COUNTS AS A RESULT

Contribution: a better ratio, a faster algorithm at the same ratio, or an inapproximability bound showing no better ratio exists.

BARGAIN 2

Heuristics

Give up the guarantee entirely; shift the whole burden onto experiment.

WHAT COUNTS AS A RESULT

Contribution: a method that wins on accepted instance libraries, against tuned competitors, with Lecture 2's empirical obligations in full.

BARGAIN 3

Parameterisation

Keep exactness; confine the exponential to a parameter small in practice.

WHAT COUNTS AS A RESULT

Contribution: an $f(k) \cdot \text{poly}(n)$ algorithm, a polynomial kernel, or a proof that no such kernel exists.

BARGAIN 4

Restricted inputs

Keep exactness and efficiency; narrow the problem instead.

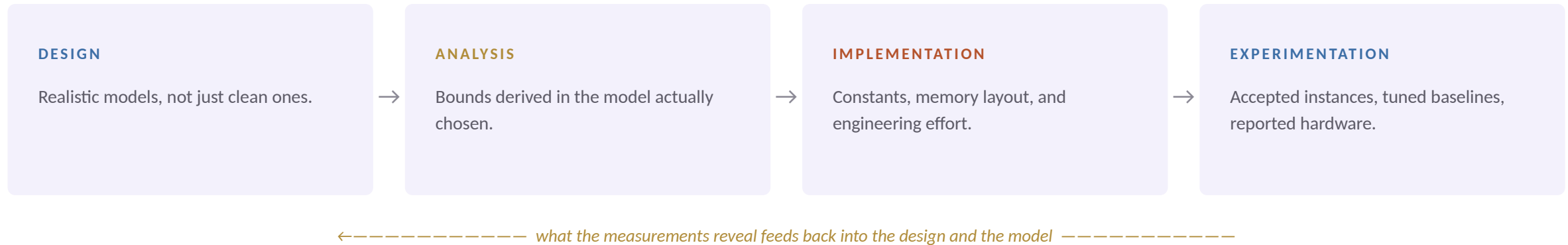
WHAT COUNTS AS A RESULT

Contribution: tractability on planar graphs, bounded treewidth, or whatever structure real instances actually have.

Each is a different bargain with the same hardness result. Naming which one you struck is the first sentence of your contribution — and the thing an unfocused paper never manages to say.

Algorithm Engineering: Closing the Loop

The pipeline becomes a cycle. Measurement is allowed to refute not just the code, but the model the analysis was carried out in.



Measurement can refute the model

Finding that a “linear” routine is memory-bound is not an implementation detail. It says the RAM model priced the wrong resource, and the analysis has to be redone in a model that counts block transfers.

Shared instances make results comparable

TSPLIB, the DIMACS Implementation Challenges, SATLIB and SNAP let a claim made in 2008 still be checked against one made today. Self-generated instances cannot do this.

The empirical half inherits every obligation

The moment the claim is “faster in practice”, it is an inductive claim, and Lecture 2’s rules apply in full: tuned baselines, repetitions, dispersion, and a stated instance population.

Worked Example: An Algorithmic Contribution, End to End

Claim under test: “Point-to-point shortest-path queries on continental road networks can be answered in microseconds after preprocessing.” — *contraction hierarchies*, Geisberger et al., 2008

THE FORMALISATION

Given a directed graph with non-negative edge weights, fixed in advance, answer many s–t distance queries.

Preprocessing time is amortised over the query stream, so it is a separate budget — a modelling choice that makes the whole approach possible.

THE SPLIT THAT MATTERS

Correctness is deductive and unconditional. The speed-up is inductive and conditional on road networks having highway-like structure.

A paper that reported both as though they were one claim would invite exactly the objection Lecture 2’s triangulation table predicts: a proof offered where a benchmark was needed, or the reverse.

1

Design

Contract nodes one at a time in a heuristically chosen order, inserting shortcut edges wherever a removed node lay on the only shortest path between two neighbours.

2

Analyse

Prove that contraction preserves all shortest-path distances, so a bidirectional search over upward edges only is still exact. Correctness holds for every input graph.

3

Implement

The achieved speed-up is dominated by engineering: the contraction order, the priority queue, and the memory layout of the shortcut graph.

4

Experiment

Measure on the standard continental road networks used by the DIMACS shortest-path challenge, against plain Dijkstra and against earlier speed-up techniques.

The Object of Study Contains People

Software-engineering research studies the construction and evolution of software by people, using tools and processes, under organisational constraints.

- **Human variance dwarfs most effects you are looking for.**

Individual developer productivity varies by a large factor on the same task. A 20% difference is decisive for an algorithm and easily noise across developers — which is why sample size and dispersion matter more here, not less.

- **You cannot hold the world fixed and keep it realistic.**

Laboratory experiments buy control and lose realism; field studies do the reverse. Neither is the better method — the choice is a design decision that must be stated and defended.

- **Threats to validity are the centre of the paper.**

Because the dominant mode is empirical and the dominant inference is induction, the argument lives or dies on the population and the constructs. That section is not an appendix here.

The reason SE research feels harder than it looks: the interesting questions are causal, and the available data is almost always observational.

THE SAME SLOT, DIFFERENT CONTENTS

In algorithmic work, the risky premise is the model.

In SE work, it is the population.

Both occupy the position Lecture 1 called the warrant: the unstated reason the evidence supports the claim. Making it explicit is the same discipline in both traditions — it just has a different name in each.

The Empirical Methods Toolkit

Six standard designs. They differ mainly in what they sacrifice — control, realism, scale, or the ability to say anything causal.

Controlled experiment

Assign participants to treatments and measure an outcome on a fixed task. Highest internal validity, lowest external: the tasks small enough to run are rarely the work you care about.

Case study

Deep investigation of one or a few real settings, with a stated unit of analysis and several data sources. High realism, limited and carefully argued generalisation.

Survey

A designed instrument administered to a sampled population of practitioners. Good for prevalence and attitude; weak for causal claims. Report the sampling frame and response rate.

Repository mining

Observational data at scale from version control, issue trackers and CI logs. Enormous samples, no control at all, and heavy selection bias toward public projects.

Observational study

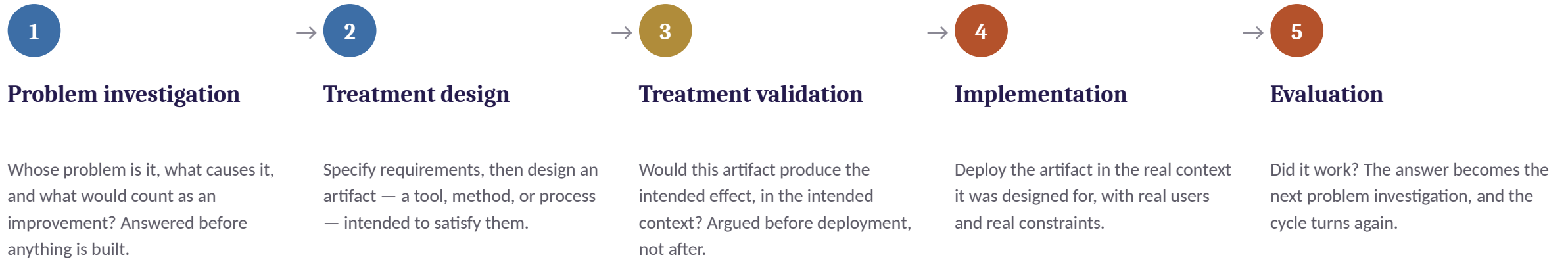
Watching practitioners in situ, including ethnographic work. Strongest for discovering constructs and generating hypotheses that other designs then test.

Design science

Build an artifact to solve a real problem in a real context, evaluate it there, and report the design knowledge that transfers beyond the one case.

Design Science: When the Contribution Is an Artifact

Wieringa's engineering cycle, the standard framing for thesis work that builds something to solve a real problem.



THE DISTINCTION THAT ORGANISES A THESIS

Wieringa separates design problems — “how do we build something that achieves X?” — from knowledge questions — “is it true that Y?”. They call for different evidence and belong in different chapters: a design problem is answered by an artifact plus a validation argument, a knowledge question by a study. Most confused theses are confused because these two were attempted in the same chapter.

Every Software Metric Is a Proxy

Lecture 2 called this construct validity. In SE research it is the threat that sinks the most papers, because the metric is easy to compute and the construct is hard to define.

METRIC	THE CONSTRUCT IT IS MEANT TO CAPTURE	WHY THE CONSTRUCT IS CONTESTED
Cyclomatic complexity	Comprehension difficulty and testing effort.	Correlates so strongly with size that most studies find it adds little beyond counting lines.
Defect density	Product quality.	Depends on how hard anyone looked. More testing produces more recorded defects, not worse software.
Code churn	Instability and change risk.	Cannot distinguish thrashing from healthy, active development on a component under deliberate revision.
Coupling and cohesion	Maintainability and modularity.	Definitions differ between tools, so absolute values are not comparable across studies or languages.
Lines of code	Size, effort, and productivity.	Language- and style-dependent; as a productivity measure it actively rewards writing more code.

None of this makes the metrics useless. It makes them proxies that must be named as proxies — and defended, whenever the claim is about the construct rather than the number.

Working With Human Participants

Four obligations that apply the moment your data comes from people rather than machines.

OBLIGATION 1

Sampling

Who are the participants, and who exactly do they stand for?

IN PRACTICE

Students are a defensible proxy for junior developers on small, well-specified tasks — and a poor one for architectural judgement or long-horizon maintenance.

OBLIGATION 2

Tasks

Is the task short enough to run also representative of the work you claim about?

IN PRACTICE

Usually not. State the trade explicitly rather than letting the abstract quietly generalise from a two-hour exercise to professional practice.

OBLIGATION 3

Statistical power

Can this design detect the effect you care about, if it exists?

IN PRACTICE

Developer performance varies enormously between individuals. Small n plus large variance makes most negative results uninformative — compute power before running, not after.

OBLIGATION 4

Ethics

Consent, anonymisation, the right to withdraw, and institutional approval.

IN PRACTICE

Mining public repositories still involves identifiable people and their work. Unit III returns to this in detail; treat it as binding, not procedural.

The participants are not instruments. Every one of these obligations is simultaneously a methodological requirement and an ethical one.

Worked Example: An SE Study, Read as an Argument

Claim under test: “Test-driven development improves external code quality without reducing productivity.” — *one of the most replicated claims in empirical SE*

COMPONENT	HOW THE STUDY SUPPLIES IT
Question type	A knowledge question, comparative and causal — which forces a controlled design rather than repository mining.
Design	Participants assigned to test-first or test-last on the same task; defect count and completion time measured.
Population	Whoever the participants stand for: usually students, or the developers of one company. This is where the qualifier must live.
Constructs	“External quality” operationalised as failures against a hidden acceptance suite; “productivity” as time to completion.
Threats	Internal: did the test-first group actually write tests first? External: task size and duration. Construct: a hidden suite measures conformance, not quality.
Outcome	Across many replications and systematic reviews the effect is small and inconsistent, with process conformance a stronger moderator than TDD itself.

That is a real result, and a publishable one. A field that reports only confirmations is not, by Lecture 1's Popperian standard, testing anything at all.

Shared Instances and Datasets

Both traditions depend on data nobody in the paper chose. These are the collections that make results comparable across groups and across years.

TSPLIB • DIMACS Challenges

Curated instances for routing, shortest paths, graph partitioning and matching, with published reference results from the implementation challenges.

SATLIB • SAT Competition

An annual competition where anyone may submit adversarial instances — an unusually honest arrangement, and a major driver of solver progress.

SNAP • Network Repository

Large real-world graphs — social, web, citation, road — so that graph-algorithm claims are tested on structures that actually occur.

Defects4J • BugsInPy

Reproducible real bugs with tests, for fault-localisation and automated program-repair research. Reproducibility is the entire point of the dataset.

SPEC CPU • DaCapo

Standard workloads for systems, compiler and runtime measurement, with strict published rules on how results may be reported.

PROMISE • SEACRAFT

Curated software-engineering datasets for defect prediction and effort estimation, assembled precisely so studies can be re-run and compared.

Artifact Evaluation and Reproducibility

The ACM badges, in the order of increasing demand. Knowing what each one actually certifies tells you what to claim — and what not to.

1

Artifacts Available

The artifact is placed in a permanent public archive with a DOI. Says nothing about whether it works.

2

Evaluated — Functional

Documented, consistent, complete, and exercisable: a reviewer got it to run and produce something.

3

Evaluated — Reusable

All of the above, plus structured well enough for someone else to build on beyond this paper.

4

Results Reproduced

A different team obtained the same results using the authors' own artifacts.

5

Results Replicated

A different team obtained the same results without the authors' artifacts, from the description alone.

A TERMINOLOGY TRAP WORTH KNOWING

ACM swapped the meanings of “reproduced” and “replicated” in 2020 to align with the NISO terminology. Papers published before then use the two words the other way round. When you use either term in your own writing, say which definition you mean.

WHY THIS IS METHODOLOGY, NOT ADMIN

Lecture 2 argued that a demonstration is only admissible evidence when it is re-runnable. Badging is that argument made institutional: it converts “trust us” into something a stranger can check, in both traditions at once.

The Same Five Threats, in Two Dialects

Read across each row: the traditions use different vocabulary for what is structurally the same methodological failure.

THE THREAT	IN ALGORITHMIC WORK	IN SOFTWARE ENGINEERING WORK
The risky premise	The cost model: unit-cost memory, an assumed input distribution, a machine that does not exist.	The population: which projects, which developers, which tasks the sample is taken to stand for.
Tuning to the test set	Parameters and heuristics tuned on the very instances used to report the results.	The model, metric or subgroup selected after seeing which one separated the groups.
The weak baseline	Comparing an engineered implementation against a textbook one, and reporting the engineering as the idea.	Comparing a new tool against no tool at all, or against a deliberately naive process.
Generalisation	Instance families chosen — or generated — because the method happens to suit them.	Public repositories and student participants standing in, unqualified, for professional practice.
The standard defence	Accepted instance libraries, released code, and the machine, compiler and flags reported.	A defended sampling frame, a published protocol, held-out replication, and available data.

The symmetry is the point: if you can diagnose one tradition's methodology, you already have most of what you need to read the other's.

Choosing an Approach for Your Own Question

Find the question you are actually asking, then read across. The last column is what taking that route commits you to.

IF YOUR QUESTION IS...	APPROACH	PRIMARY EVIDENCE	THE OBLIGATION IT BRINGS
“Can this be solved faster, provably?”	Design and analysis	A proof inside a stated model	Show the bound is tight, or say plainly that it is not
“Is my algorithm faster on real inputs?”	Algorithm engineering	Measurement on accepted instances	Tuned baselines, released code, reported hardware
“Does this technique help developers?”	Experiment or case study	Human-subject data, threats stated	Ethics approval, power analysis, a defined population
“What do developers actually do?”	Repository mining	Observational data at scale	A defended sampling frame — and no causal language
“Does this tool solve a real problem?”	Design science	An artifact plus evaluation in context	Separate the design problem from the knowledge question

Nothing here forbids combining routes — most strong theses do. What it forbids is answering a question from one row with the evidence from another.

Failure Modes of Each Tradition

Three from each side. Every one is common enough that reviewers have a stock phrase ready for it.

The unanalysed algorithm

A new procedure with no correctness argument and no derived bound, evaluated only on instances the authors generated themselves.

The self-selected instance family

Benchmark instances chosen, or synthesised, because the proposed method happens to exploit their structure.

The textbook baseline

An engineered implementation compared against a naive one, so that ordinary engineering effort is reported as the scientific contribution.

The demo as evaluation

The authors running their own tool on examples they picked, presented as evidence that it helps anyone other than the authors.

Students as industry

A short laboratory study on undergraduates whose abstract states its conclusion about professional software developers.

Causal language on observational data

“Adopting X reduces defects”, drawn from repository mining that could only ever have shown a correlation.

A Checklist for Your Own Study

- ✓ **Name the tradition.** Is the object of study a problem, an artifact, or a practice? Every later methodological decision follows from that answer, so make it explicitly.
- ✓ **State the model or the population.** An algorithmic paper's cost model and an SE paper's sampling frame occupy the same slot in the argument. Whichever you have, put it where a reader will find it.
- ✓ **Analyse before you measure, and measure after you analyse.** A bound with no implementation and an implementation with no bound are each half a contribution. Say which half you are offering if you only have one.
- ✓ **Use instances and baselines you did not choose.** Accepted libraries, the tuned state of the art, and somebody else's data wherever it exists. Self-chosen everything is the fastest route to rejection.
- ✓ **Separate design problems from knowledge questions.** “How do we build X” and “is it true that Y” need different evidence and different chapters. Mixing them is the commonest structural fault in a thesis.
- ✓ **Release the artifact, and claim only the badge it would earn.** Available, functional, reusable — or none. Claim only what a stranger with your repository could actually verify.

Discussion

In pairs, ten minutes. Bring one answer back for class critique.

- 1 Take your own project or thesis topic. Is its object of study a problem, an artifact, or a practice — and does the evaluation you have planned match that answer?
- 2 Find an algorithms paper with an experimental section. Where did its instances come from, who tuned the baseline, and could you re-run any of it from what is written?
- 3 Find an SE paper with a human-subject study. What population do the participants stand for, and does the abstract respect that boundary or quietly cross it?
- 4 Pick a widely repeated claim in your area that you have never seen replicated. Design the cheapest study that could overturn it — and say what result would.

Key Takeaways

- 1 Computing science research runs in two dominant traditions. The algorithmic studies a problem, and its risky premise is the cost model; the software-engineering studies a socio-technical practice, and its risky premise is the population.
- 2 The algorithmic pipeline runs formalise, design, analyse, implement, experiment — and algorithm engineering closes it into a loop, where measurement can refute the model itself and not merely the code.
- 3 Once a problem is proved hard, four honest bargains remain: approximation, heuristics, parameterisation, and restricted inputs. Naming the one you struck is the first sentence of your contribution.
- 4 Empirical SE must trade control against realism and cannot have both; its methods differ mainly in which they sacrifice. Every software metric is a proxy, so construct validity is the standing threat.
- 5 Shared instance libraries, tuned baselines, and artifact badging are what make either tradition's claims comparable across groups and years — and they are the cheapest thing you can do to make your own work checkable.

Further Reading

Skiena, S. S. The Algorithm Design Manual. 3rd ed., Springer, 2020. — the course's listed reference for algorithmic method.

Sanders, P. “Algorithm Engineering — An Attempt at a Definition.” Efficient Algorithms, LNCS 5760, Springer, 2009.

McGeoch, C. C. A Guide to Experimental Algorithmics. Cambridge University Press, 2012.

Williamson, D. P. & Shmoys, D. B. The Design of Approximation Algorithms. Cambridge University Press, 2011.

Cygan, M. et al. Parameterized Algorithms. Springer, 2015.

Wohlin, C. et al. Experimentation in Software Engineering. Springer, 2012. — the standard reference for SE experiments.

Runeson, P. & Höst, M. “Guidelines for Conducting and Reporting Case Study Research in Software Engineering.” Empirical Software Engineering, 2009.

Wieringa, R. J. Design Science Methodology for Information Systems and Software Engineering. Springer, 2014.

Geisberger, R., Sanders, P., Schultes, D. & Delling, D. “Contraction Hierarchies: Faster and Simpler Hierarchical Routing in Road Networks.” WEA, LNCS 5038, 2008.